Documentação CRUD — Usuários (API REST)

Documentação completa do CRUD de **Usuários**, demonstrando cada endpoint da API REST rodando em `http://localhost:3000/usuarios`, incluindo os testes de sucesso e de erro. Todas as requisições foram testadas via **Postman**.

Visão Geral

Método	Rota	Descrição	Status esperado
POST	`/usuarios`	Criar um novo usuário	`201`
GET	`/usuarios`	Listar todos os usuários	`200`
GET	`/usuarios/:id`	Buscar usuário por ID	`200`
PUT	`/usuarios/:id`	Atualizar usuário existente	`200`
DELETE	`/usuarios/:id`	Deletar usuário por ID	`200`

Tecnologias Utilizadas

- **Runtime:** Node.js
- **Framework:** Express.js
- **Banco de dados:** MySQL (via `mysql2/promise`)
- **Ferramenta de testes:** Postman

Estrutura dos Campos

Campo	Tipo	Obrigatório	Descrição
`tipoUsuario`	INT	Sim	Tipo/perfil do usuário
`nomeUsuario`	VARCHAR	Sim	Nome completo do usuário
`email`	VARCHAR	Sim	Email do usuário
`senha`	VARCHAR	Sim	Senha do usuário
`contato`	VARCHAR	Não	Telefone de contato

Testes no Postman

1. POST — Criar Usuário

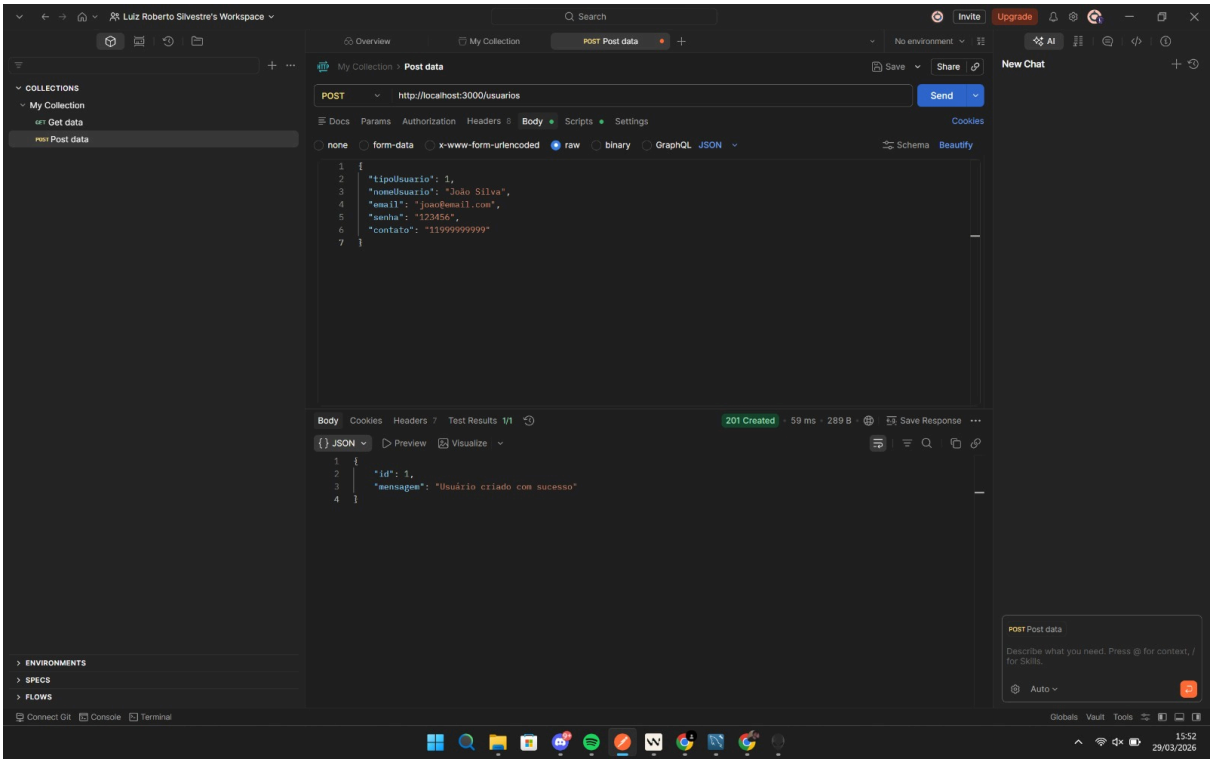
- **Método:** `POST`
- **URL:** `http://localhost:3000/usuarios`
- **Body (JSON):**

```
{
  "tipoUsuario": 1,
  "nomeUsuario": "João Silva",
  "email": "joao@email.com",
  "senha": "123456",
  "contato": "11999999999"
}
```

- **Resposta esperada:** Status `201`

```
{
  "id": 1,
  "mensagem": "Usuário criado com sucesso"
}
```

Print do teste no Postman:



Print — POST Criar Usuário

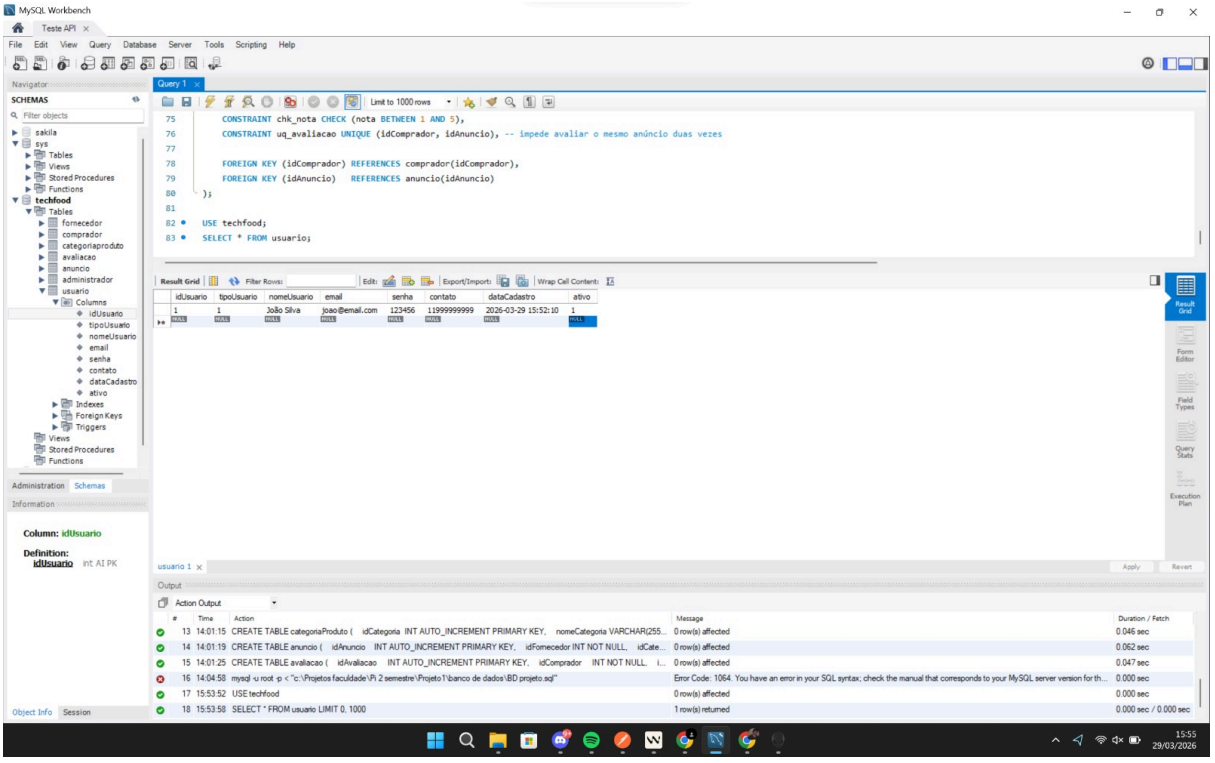
---

2. GET — Listar Todos os Usuários

- **Método:** `GET`
- **URL:** `http://localhost:3000/usuarios`
- **Body:** Nenhum
- **Resposta esperada:** Status `200` + array com os usuários cadastrados

```
[
  {
    "tipoUsuario": 1,
    "nomeUsuario": "João Silva",
    "email": "joao@email.com",
    "senha": "123456",
    "contato": "11999999999"
  }
]
```

Print do teste no Postman:



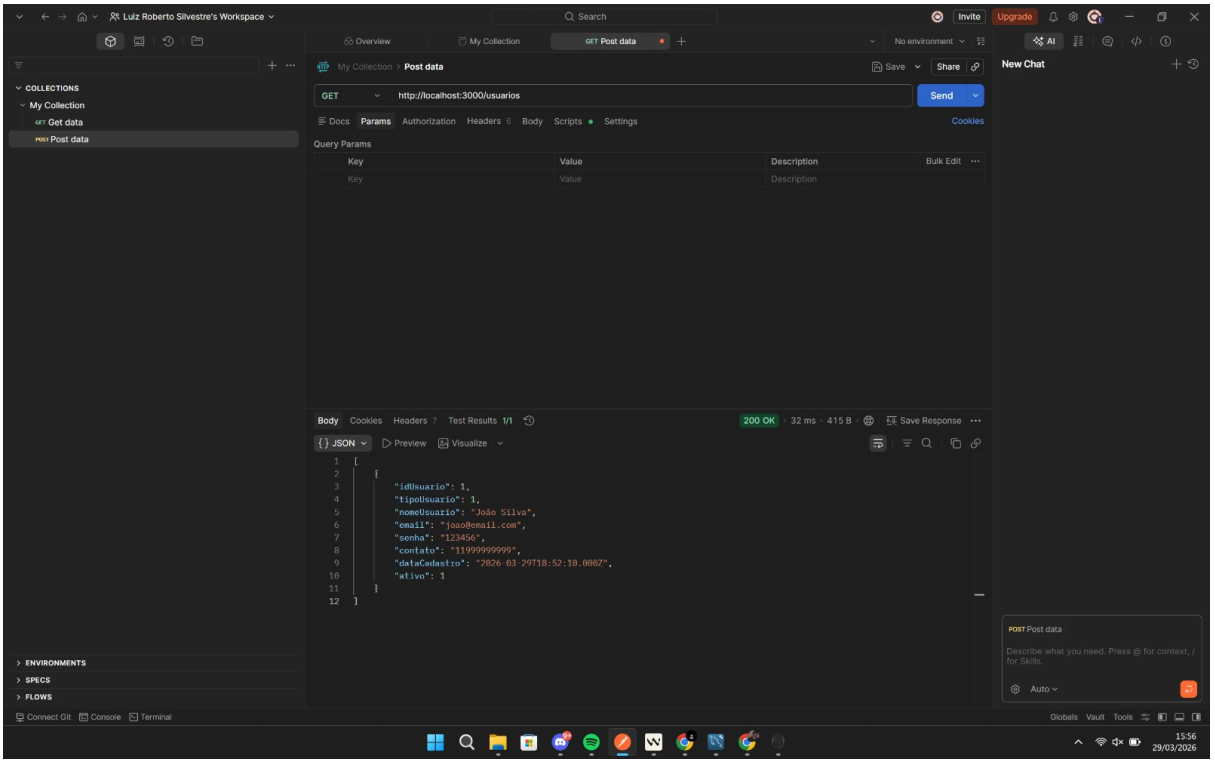
Print — GET Listar Todos

---

3. GET — Buscar Usuário por ID

- **Método:** `GET`
- **URL:** `http://localhost:3000/usuarios/1`
- **Body:** Nenhum
- **Resposta esperada:** Status `200` + dados do usuário com ID 1

Print do teste no Postman:



Print — GET por ID

---

4. PUT — Atualizar Usuário

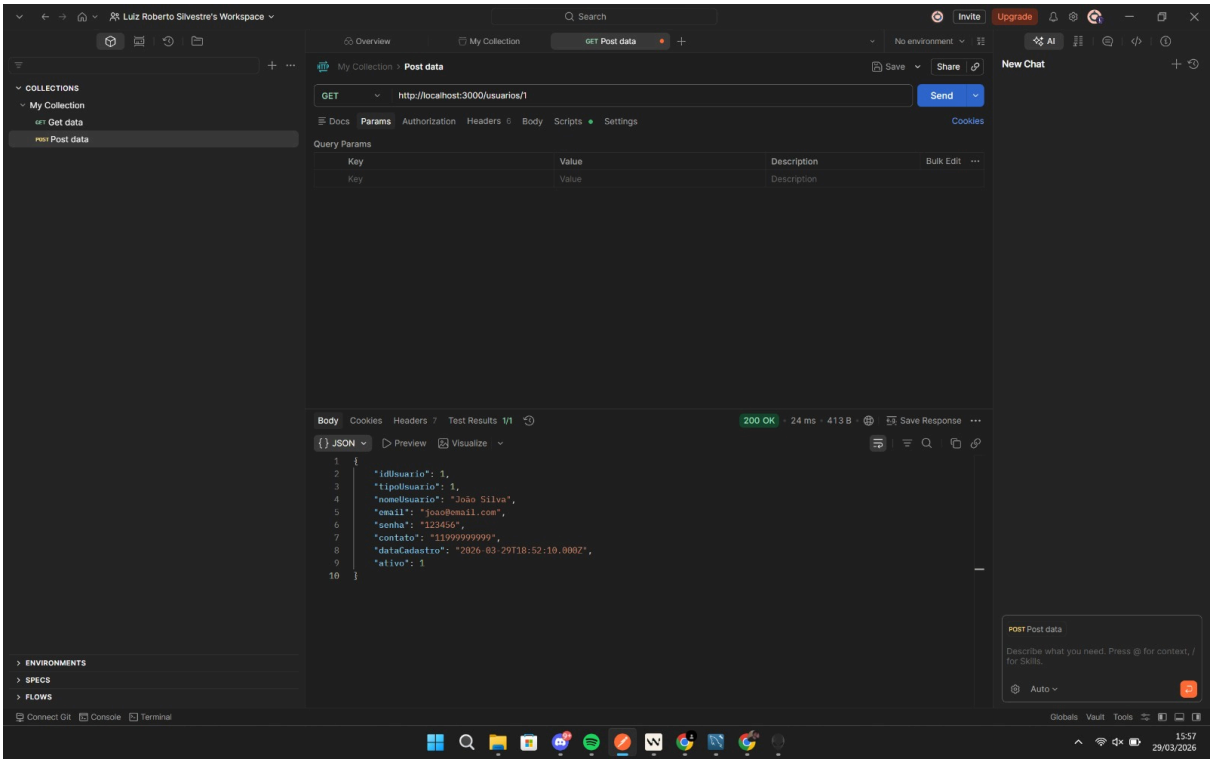
- **Método:** `PUT`
- **URL:** `http://localhost:3000/usuarios/1`
- **Body (JSON):**

```
{
  "tipoUsuario": 1,
  "nomeUsuario": "João Silva Atualizado",
  "email": "joao.novo@email.com",
  "senha": "654321",
  "contato": "11888888888"
}
```

- **Resposta esperada:** Status `200`

```
{
  "mensagem": "Usuário atualizado com sucesso"
}
```

Print do teste no Postman:



Print — PUT Atualizar

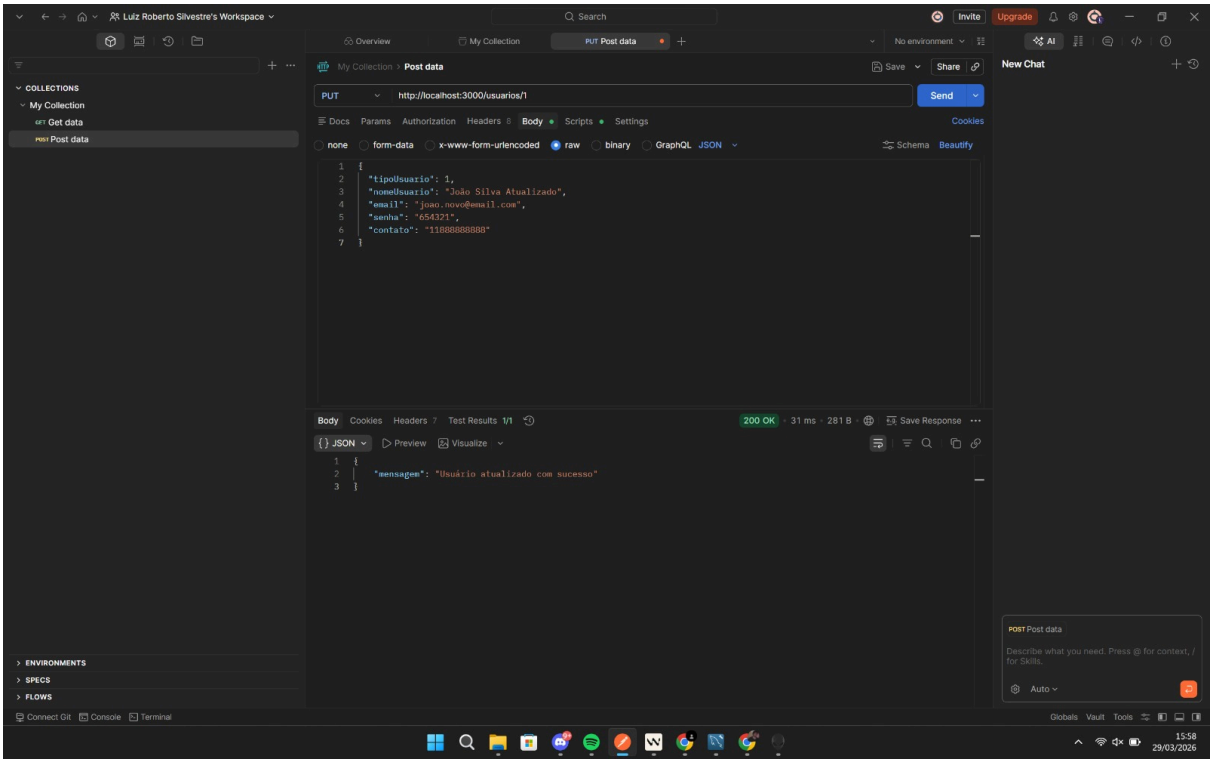
---

5. DELETE — Deletar Usuário

- **Método:** `DELETE`
- **URL:** `http://localhost:3000/usuarios/1`
- **Body:** Nenhum
- **Resposta esperada:** Status `200`

```
{
  "mensagem": "Usuário deletado com sucesso"
}
```

Print do teste no Postman:



Print — DELETE Usuário

Testes de Erro

6. ERRO — POST sem campos obrigatórios

- **Método:** `POST`
- **URL:** `http://localhost:3000/usuarios`
- **Body (JSON):** Enviado com campos faltando

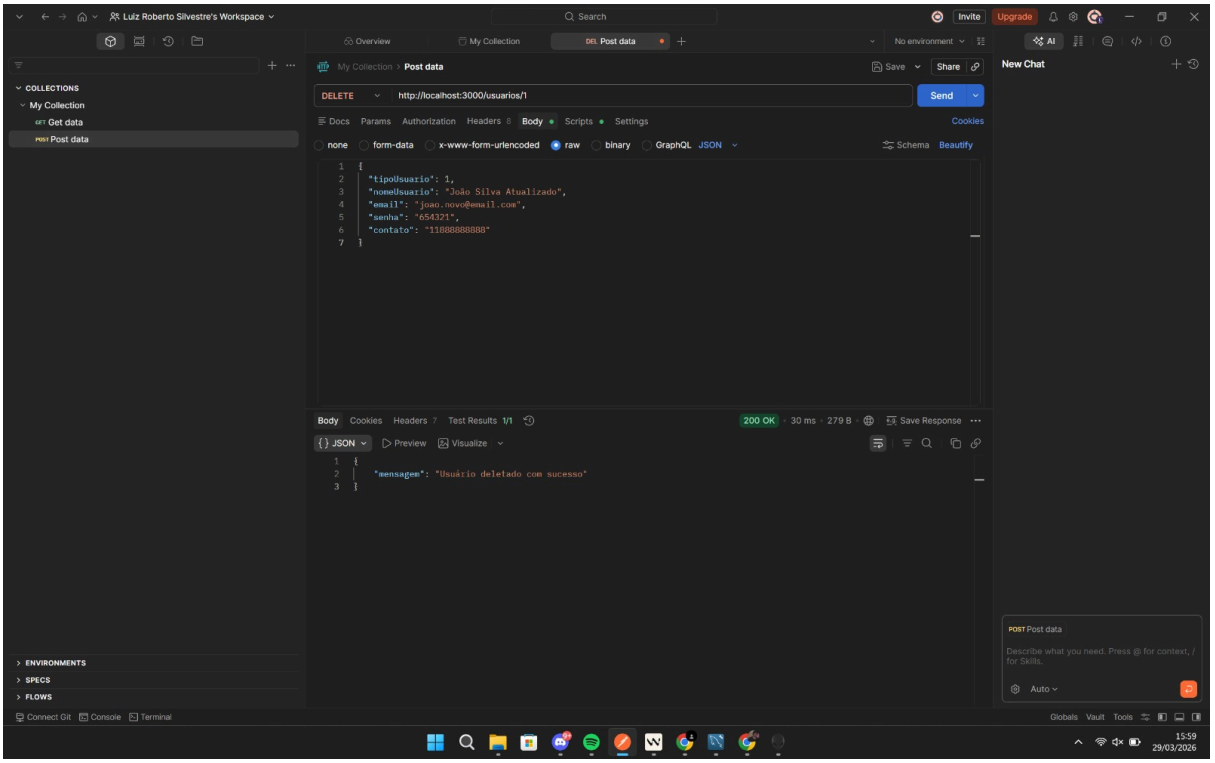
```
{
  "tipoUsuario": 1,
  "nomeUsuario": "Teste"
}
```

- **Resposta esperada:** Status `400`

```
{
  "erro": "Campos obrigatórios: tipoUsuario, nomeUsuario, e
```

```
mail, senha"
}
```

Print do teste de erro no Postman:



Print — ERRO POST campo faltando

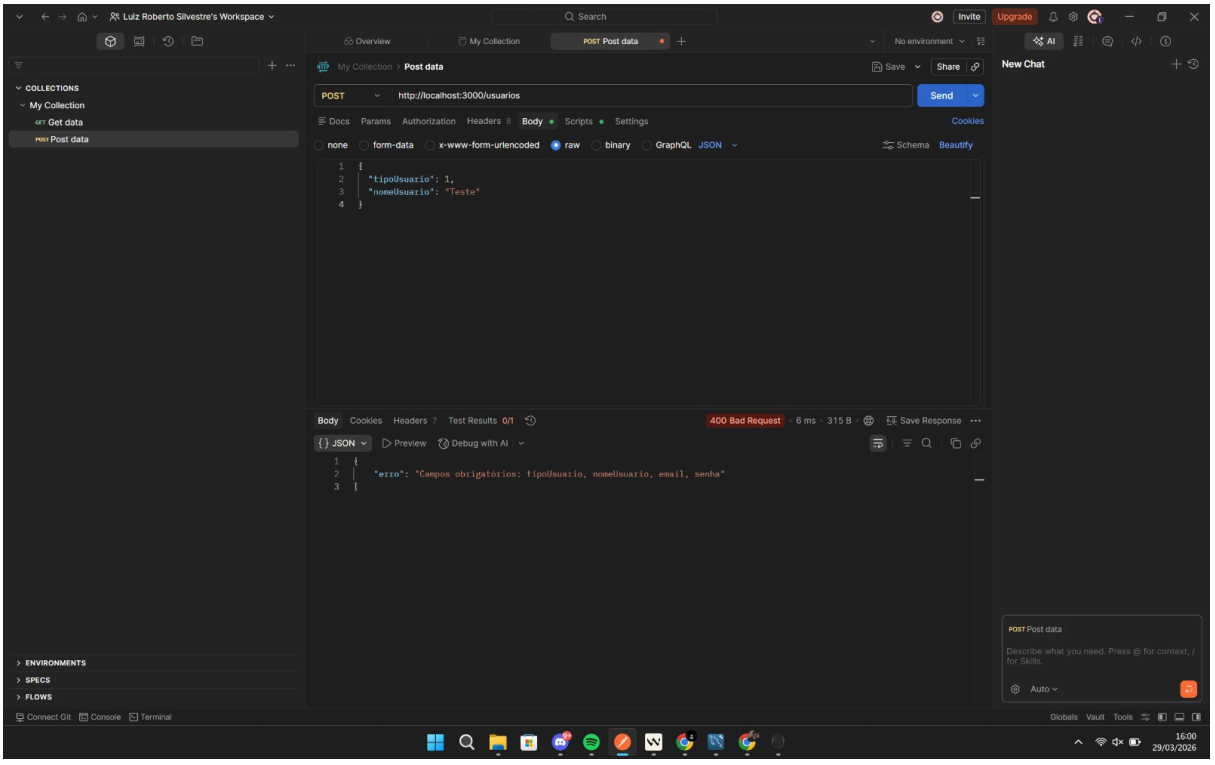
---

7. ERRO — GET com ID inexistente

- **Método:** `GET`
- **URL:** `http://localhost:3000/usuarios/9999`
- **Body:** Nenhum
- **Resposta esperada:** Status `404`

```
{
  "erro": "Usuário não encontrado"
}
```

Print do teste de erro no Postman:



Print — ERRO GET ID inexistente

Resumo dos Testes

#	Teste	Método	Status	Resultado
1	Criar usuário	`POST`	`201`	Sucesso
2	Listar todos	`GET`	`200`	Sucesso
3	Buscar por ID	`GET`	`200`	Sucesso
4	Atualizar usuário	`PUT`	`200`	Sucesso
5	Deletar usuário	`DELETE`	`200`	Sucesso
6	POST sem campo obrigatório	`POST`	`400`	Erro esperado
7	GET com ID inexistente	`GET`	`404`	Erro esperado